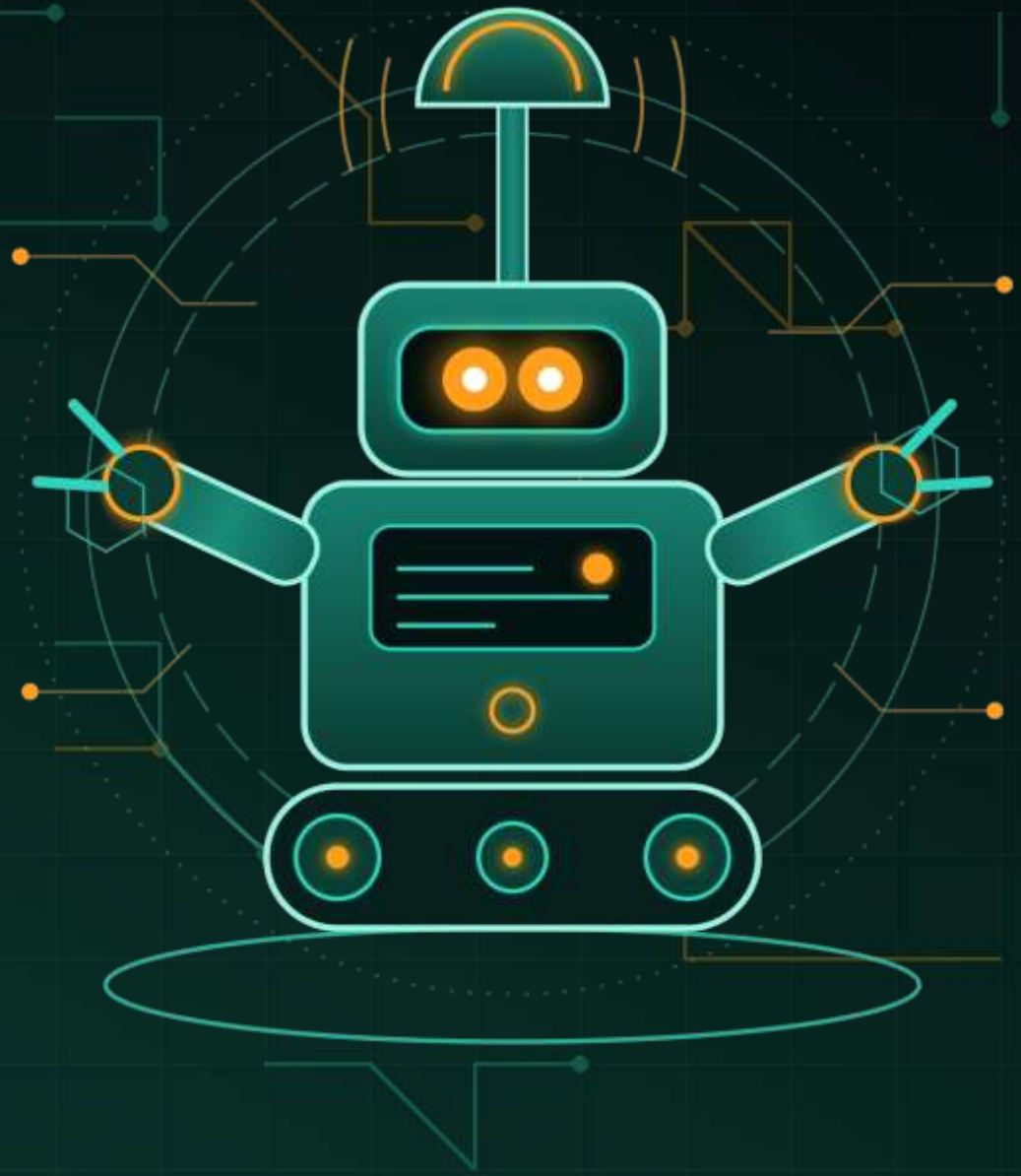
VSP TECHVERSE
An Institute of AI & Robotics

Python Classes for AI & Robotics



Classes · Objects · Inheritance — Python concepts with AI & Robotics examples

VSP TECHVERSE
An Institute of AI & Robotics

class

Classes & Objects

A class is a blueprint for creating objects such as robots, sensors and drones.

TOPICS IN THIS SECTION

1. Python Classes
2. Create & Delete Objects
3. Multiple Objects
4. The pass Statement
5. The __init__() Method
6. The __str__() Method
7. Object Methods
8. The self Parameter
9. Modify Properties
10. Delete Properties

Python Classes

example.py

```
class Robot:
  battery = 100

print(Robot.battery)
```

OUTPUT

```
100
```

KEY POINTS

- Python is object-oriented: almost everything is an object with properties and methods
- A class works as a blueprint for creating objects
- Use the class keyword to define one, with properties inside

KEY NOTE

A class is a blueprint; an object is one thing built from it.

Create & Delete Objects

example.py

```
class Robot:
  battery = 100

r1 = Robot()
print(r1.battery)

del r1
```

OUTPUT

```
100
```

KEY POINTS

- Call the class name to create an object
- Read a property with dot notation
- The del keyword deletes an object

KEY NOTE

The del keyword deletes objects.

Multiple Objects

example.py

```
class Robot:
  battery = 100

r1 = Robot()
r2 = Robot()
r3 = Robot()

print(r1.battery)
print(r2.battery)
print(r3.battery)
```

OUTPUT

```
100
100
100
```

KEY POINTS

- Create as many objects from one class as you need
- Each object is independent
- All start with the class property values

KEY NOTE

Each object is independent and has its own copy of the properties.

The pass Statement

example.py

```
class Drone:
  pass

d1 = Drone()
print(type(d1))
```

OUTPUT

```
<class '__main__.Drone'>
```

KEY POINTS

- A class definition cannot be empty
- pass is a placeholder for a class with no content yet
- Useful when sketching a design before adding code

KEY NOTE

Use pass when a class has no content yet.

The __init__() Method

example.py

```
class Robot:
  def __init__(self, name, battery):
    self.name = name
    self.battery = battery

r1 = Robot("Rover-2", 85)

print(r1.name)
print(r1.battery)
```

OUTPUT

```
Rover-2
85
```

KEY POINTS

- __init__() runs automatically every time an object is created
- Use it to assign values to properties
- The first parameter, self, refers to the object being created

KEY NOTE

__init__() is called automatically when the class creates a new object.

The __str__() Method

example.py

```
class Robot:
  def __init__(self, name, battery):
    self.name = name
    self.battery = battery

  def __str__(self):
    return f"{self.name} ({self.battery}%)"

r1 = Robot("Rover-2", 85)

print(r1)
```

OUTPUT

```
Rover-2 (85%)
```

KEY POINTS

- __str__() controls what print() shows for an object
- Without it, print shows only the object's memory location
- It must return a string

KEY NOTE

__str__() controls what is returned when the object is printed.

Object Methods

example.py

```
class Robot:
  def __init__(self, name, battery):
    self.name = name
    self.battery = battery

  def status(self):
    print("Robot " + self.name + " is online")

r1 = Robot("Rover-2", 85)
r1.status()
```

OUTPUT

```
Robot Rover-2 is online
```

KEY POINTS

- Objects can contain methods, which are functions that belong to the object
- Define them inside the class with def
- Call them with dot notation

KEY NOTE

Methods in objects are functions that belong to the object.

The self Parameter

example.py

```
class Robot:
  def __init__(rbt, name, battery):
    rbt.name = name
    rbt.battery = battery

  def report(abc):
    print("Battery of " + abc.name + ":", abc.battery)

r1 = Robot("Rover-2", 85)
r1.report()
```

OUTPUT

```
Battery of Rover-2: 85
```

KEY POINTS

- self is a reference to the current object
- It gives access to the object's properties and methods
- It does not have to be named self, but it must be the first parameter

KEY NOTE

The name self is a convention; any name works as long as it is first.

Modify Properties

example.py

```
class Robot:
  def __init__(self, name, battery):
    self.name = name
    self.battery = battery

r1 = Robot("Rover-2", 85)

r1.battery = 40

print(r1.battery)
```

OUTPUT

```
40
```

KEY POINTS

- Change an object's property by assigning a new value
- Other objects of the class are not affected

KEY NOTE

You can modify properties on objects.

Delete Properties

example.py

```
class Robot:
  def __init__(self, name, battery):
    self.name = name
    self.battery = battery

r1 = Robot("Rover-2", 85)

del r1.battery

print(r1.name)
```

OUTPUT

```
Rover-2
```

KEY POINTS

- The del keyword removes a property from an object
- Only that object loses the property
- Delete the object itself with del object

KEY NOTE

You can delete properties on objects by using the del keyword.

is-a

Inheritance

Inheritance lets a child class reuse the properties and methods of a parent class.

TOPICS IN THIS SECTION

1. Python Inheritance
2. Create a Child Class

Python Inheritance

example.py

```
class Robot:
  def __init__(self, name, model):
    self.name = name
    self.model = model

  def intro(self):
    print(self.name, self.model)

x = Robot("Rover", "R-1")
x.intro()
```

OUTPUT

```
Rover R-1
```

KEY POINTS

- Inheritance lets a class take on the methods and properties of another class
- The parent (base) class is the one inherited from
- The child (derived) class is the one that inherits

KEY NOTE

Any class can be a parent class.

Create a Child Class

example.py

```
class Robot:
  def __init__(self, name, model):
    self.name = name
    self.model = model

  def intro(self):
    print(self.name, self.model)

class Rover(Robot):
  pass

x = Rover("Rover", "R-2")
x.intro()
```

OUTPUT

```
Rover R-2
```

KEY POINTS

- Pass the parent class as a parameter to create a child
- The child gets all the parent's properties and methods
- Use pass when the child needs nothing extra

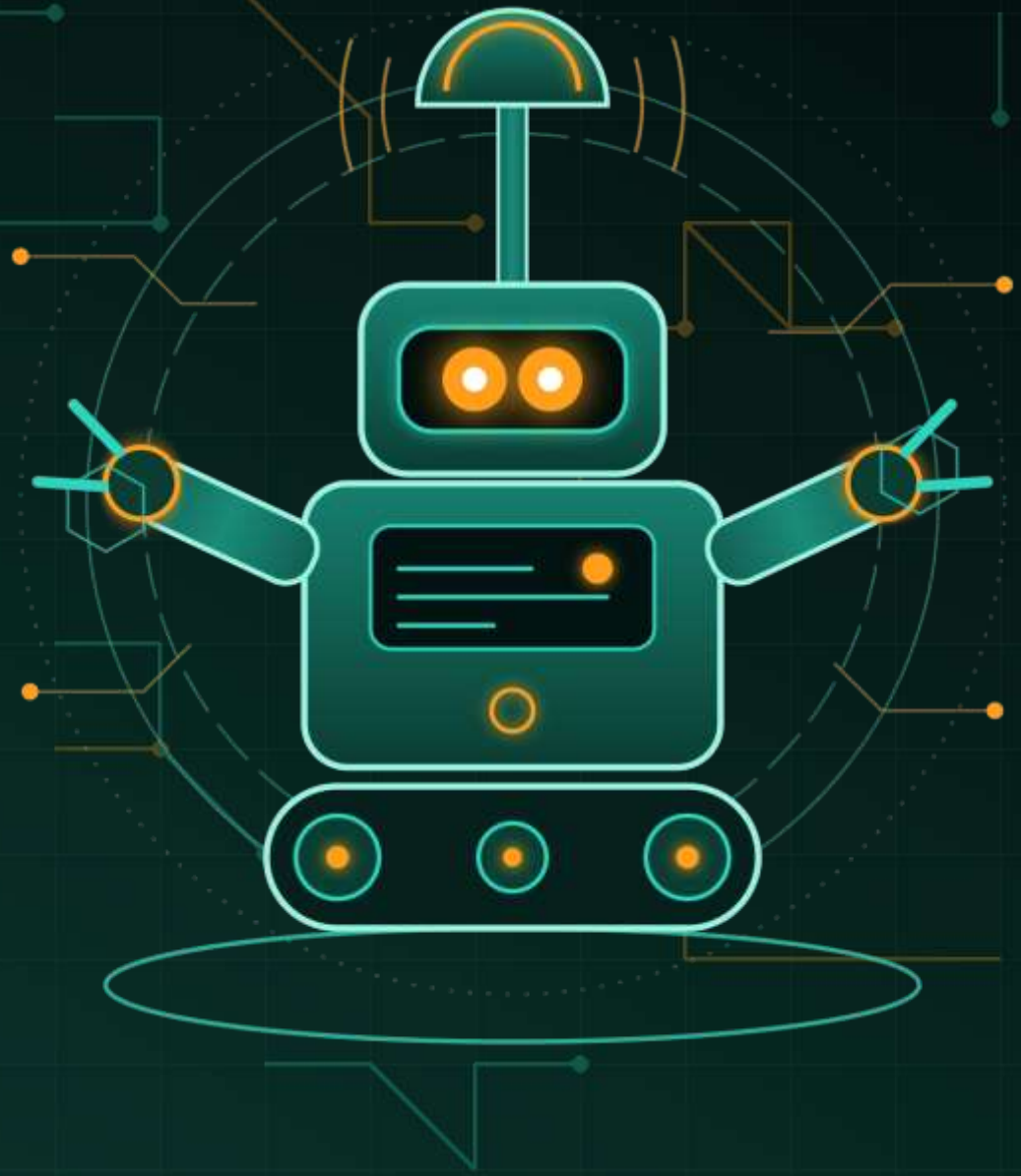
KEY NOTE

To create a child class, send the parent class as a parameter.

VSP TECHVERSE
An Institute of AI & Robotics

Thank You

Python OOP concepts, with examples adapted for AI & Robotics



VSP TECHVERSE
An Institute of AI & Robotics